

A PROJECT REPORT ON

CAMPUS SAFE

Campus Safety & Support System

A Python Console Application for Emergency Reporting, Lost & Found Tracking, and Campus Complaint Management.

STUDENT SUBMISSION DETAILS

Submitted By: **Debraj Roy**

Registration No.: **26BCE10816**

Course: **B.Tech - Computer Science**

Academic Year: **First Year**

Subject: **Programming in Python**

Session: **2026-27**

Department of Computer Science & Engineering

Submitted in partial fulfillment of the academic curriculum.

2. INTRODUCTION

Campus Safe is a simple text-based application built in Python to help manage student security and campus facility complaints in one place. On college campuses, students often lose items, notice broken utilities, or encounter emergencies. Campus Safe provides an easy-to-use menu where students can log these reports immediately.

3. PROBLEM STATEMENT

In many colleges, reporting a lost item or facility issue is done manually on paper register logs or through separate staff offices. This leads to common issues:

- **Lost Belongings:** Students have no quick way to check if someone found their lost items.
- **Emergency Delays:** Informing security guards takes time if there is no centralized desk.
- **Unsorted Complaints:** Urgent facility repairs (like electrical shocks or water leaks) get mixed up with minor issues.

Solution: A single terminal-based application that accepts, stores, prioritizes, and searches all campus safety records in real time.

4. FUNCTIONAL REQUIREMENTS

Functional requirements define what features the software performs for the user:

1. Emergency Reporting

Collects user name, emergency type, location, and description, then saves it immediately.

2. Lost & Found Tracking

Allows users to register lost items or report found belongings on campus.

3. Priority Complaint System

Automatically flags complaints as HIGH, MEDIUM, or NORMAL based on the category entered.

4. Search & Display

Provides keyword searching for lost items and displays summary reports of all entries.

5. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality and performance attributes of the software:

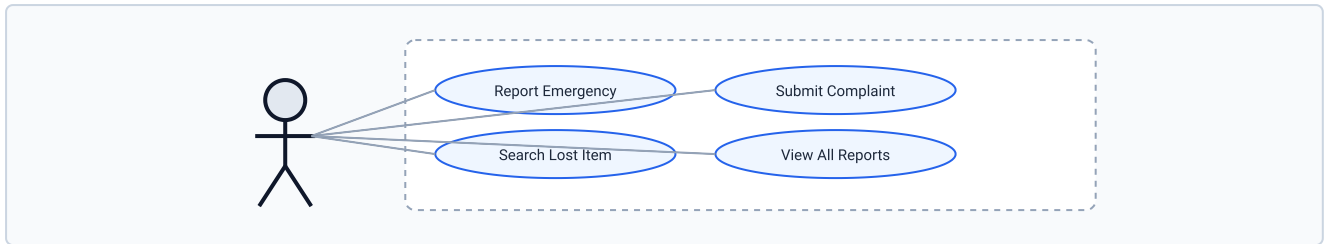
- **Ease of Use (Usability):** Simple text menu requiring no technical experience to operate.
- **Response Time (Performance):** Instant data storing and retrieval because operations run in system RAM.
- **Reliability:** Uses a continuous main program loop so the program stays open until the user explicitly chooses to exit.

6. SYSTEM ARCHITECTURE

The program follows a modular functional structure. The central control function `main_()` runs an infinite loop showing a menu, taking user input, and directing control to individual sub-functions.

7. DESIGN DIAGRAMS

A. Use Case Diagram



B. Workflow Diagram

Start → Show Menu → Input Choice
If 1-6 → Run Function → Loop Menu
If 7 → Print Goodbye → **End**

C. Sequence Diagram

User select [4] → `main_()`
`main_()` calls `submission_complaint()`
Enter Inputs → Append List → Success

D. Class / Component Diagram

Since the project uses structured functions rather than classes, components are divided by role:

- **Data Arrays:** Global 2D lists (`emergency_reported`, `items_lost`, `items_found`, `complaint`).
- **Input Handlers:** `reporting_emergency()`, `items_lost_report()`, `items_found_report()`, `submission_complaint()`.
- **Display & Search:** `viewing_all_reports()`, `searching_item_lost()`.

E. ER Diagram (Data Storage Schema)

List Name	Attributes (Fields)
<code>emergency_reported</code>	[Name, Emergency Type, Location, Description]
<code>items_lost</code>	[Name, Item Name, Last Seen Location, Description]
<code>items_found</code>	[Name, Item Name, Found Location, Description]
<code>complaint</code>	[Name, Category, Location, Details, Priority]

8. DESIGN DECISIONS & RATIONALE

- **2D Lists over Database:** Built using Python nested lists because they are easy to understand for first-year students and require no extra software setup.
- **Automated Priority Logic:** Complaints checking for categories like "security" or "electricity" automatically get assigned a `HIGH` priority tag so urgent issues stand out.

9. IMPLEMENTATION DETAILS

The system is written in standard Python 3. It utilizes four global empty lists at the start. Input functions prompt users via standard `input()` calls, create sub-lists, and append them to the main array. Search functionality utilizes case-insensitive string checking via Python's `in` operator and `.lower()` method.

10. SCREENSHOTS / RESULTS

```
=====
      CAMPUS SAFE
    Campus Safety & Support
=====
1.Report Emergency
2.Report Lost Item
3.Report Found Item
4.Submit Complaint
5.Search Lost Item
6.View All Reports
7.Exit
=====
Enter your choice: 4

--- SUBMISSION OF COMPLAINT ---
Enter your name: Alex
Enter what type of category is your complaint: Security
Enter the location: Main Gate
Enter your complaint regarding the campus: Gate light broken

Complaint is submitted successfully!
Priority: HIGH
```

11. TESTING APPROACH

The program was tested using manual functional test cases:

Test Case	Input Provided	Expected Result	Pass/Fail
Menu Option 1	Choice "1", Enter details	Emergency added to list	PASS
Priority Check	Category "electricity"	Priority printed as "HIGH"	PASS
Item Search	Search keyword "phone"	Matches "iPhone" in list	PASS
Invalid Choice	Choice "9"	Prints "Invalid choice..."	PASS

12. CHALLENGES FACED & FIXED BUGS

During testing of the source code, two main errors were found and resolved:

Bug 1: Incorrect List Name Variable (NameError)

In `viewing_emergencies()`, the loop called `range(len(emergency_reports))`, but the list was defined as `emergency_reported`. This caused the program to crash.

Fix: Corrected the variable name inside the loop to `emergency_reported`.

Bug 2: Variable Overwriting in Complaints

In `submission_complaint()`, variable `c` was used to store category and then immediately reused for complaint details. This erased the category before checking priority.

Fix: Created a separate variable `cat` for category input.

13. LEARNINGS & KEY TAKEAWAYS

- Understood how to construct multi-option menu loops using `while True` and conditional `if-elif-else` branches.
- Learned to store structured records using nested 2D lists in Python.
- Practiced debugging errors like `NameError` and variable shadowing during testing.

14. FUTURE ENHANCEMENTS

- **File Storage:** Save logs to a local file (e.g., `reports.txt`) so entries remain saved after closing the program.
- **Graphical Interface (GUI):** Build a visual window with buttons using Python's `tkinter` library.
- **Admin Password:** Add a security PIN so only authorized staff can clear or delete old reports.

15. REFERENCES

1. Python Software Foundation (2026). *Python 3 Control Flow Tools and List Documentation*. Available at: <https://docs.python.org/3/>
2. GeeksforGeeks (2026). *Python Data Structures & List Methods Tutorial*. Available at: <https://www.geeksforgeeks.org/python-programming-language/>